

Implement the Listen, Attend and Spell (LAS) model (LSTM) for Speech recognition

An attention based Deep learning model for audio transcription

Submitted to-
Prof. Pon Harshavardhanan
Associate Professor

Presented By-
Tanu Tomar- 22BAC10023
Priya Verma- 22BAC10024
Punam Thakur- 22BAC10025
Jayana Sawlani- 22BAC10042
J BalaVarun- 22BAC10044
Yash Raj Kohli- 22BEC10009

Introduction

Speech recognition is the process of converting spoken audio into written text. Traditional automatic speech recognition (ASR) systems rely on multiple components, including acoustic models, language models, and pronunciation dictionaries. These systems often require extensive feature engineering and domain expertise to build and maintain. In contrast, end-to-end approaches like the Listen, Attend and Spell (LAS) model aim to simplify this process by combining all components into a single deep learning architecture.

The LAS model uses an encoder-decoder framework with an attention mechanism that allows the system to directly learn the mapping from audio input to character-level text output. This attention-based design helps the model focus on relevant parts of the audio when predicting each letter or word. By reducing the complexity of traditional pipelines, LAS provides a more unified and flexible approach to building speech recognition systems.

Traditional ASR	End-to-End (LAS)
Multiple components	Single neural model
Needs feature engineering	Learns features automatically
Complex pipelines	Simple and unified

Objective

The primary objective of this project is to design and implement a fully functional, end-to-end speech recognition system using the Listen, Attend and Spell (LAS) model. Unlike traditional speech systems that rely on multiple separate components, this project uses a single neural network model built with PyTorch and torchaudio in Google Colab. The LAS model is trained on a subset of the LibriSpeech dataset and is capable of transcribing raw speech audio into text. The goal is to explore the effectiveness of an encoder-decoder framework with attention in recognizing and transcribing spoken English efficiently and accurately.

- Build an end-to-end ASR (Automatic Speech Recognition) model using LAS architecture.
- Eliminate the need for hand-crafted features, lexicons, and language models.
- Use PyTorch + torchaudio in Google Colab for training and testing.
- Train the model using the LibriSpeech dataset (train-clean-5 subset).
- Evaluate model performance by comparing predictions to true transcripts.

Component	Description
Dataset	LibriSpeech (train-clean-5 subset)
Input Format	.flac audio files
Output Format	Character-level transcriptions
Framework	PyTorch (Colab environment with GPU)
Model Type	Encoder-Decoder with Attention

Step 1 ;Environment setup & dependencies

To implement the LAS speech recognition model, the entire project is developed in a Google Colab environment with GPU support. The primary programming framework used is PyTorch, along with torchaudio for handling audio processing tasks. Additional libraries such as librosa (for advanced feature extraction) and jiwer (for calculating transcription error rates) are also used. These libraries are installed via pip, and their compatibility ensures smooth integration for training, evaluating, and testing the LAS model.

- ❖ Platform: Google Colab with GPU support.
- ❖ Programming Language: Python 3.11.
- ❖ Key Libraries Installed:
 - torchaudio – Audio I/O and transformations
 - librosa – Feature extraction and signal processing , for advanced feature extraction like Mel spectrograms.
 - jiwer – Evaluation metrics (WER, CER)
- ❖ Installation via: `pip install torchaudio librosa jiwer`



Tool / Library	Purpose
Google Colab	Cloud-based development with GPU
PyTorch	Deep learning framework
torchaudio	Audio loading, preprocessing, and I/O
librosa	Mel spectrograms, MFCCs, etc.
jiwer	Word Error Rate (WER) / Character Error Rate (CER)

Excel Dataset Link:
[processed_dataset \(1\).xlsx](#)

▼ Step 1: Setup & Install Dependencies

```
!pip install torchaudio librosa jiwer
```



```
Requirement already satisfied: torchaudio in /usr/local/lib/python3.11/dist-packages (2.6.0+cu124)
Requirement already satisfied: librosa in /usr/local/lib/python3.11/dist-packages (0.11.0)
Requirement already satisfied: jiwer in /usr/local/lib/python3.11/dist-packages (3.1.0)
Requirement already satisfied: torch==2.6.0 in /usr/local/lib/python3.11/dist-packages (from torchaudio) (2.6.0+cu124)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (3.18.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (4.13.1)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (3.4.2)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (2025.3.2)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (12.4.127)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (12.4.127)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (12.4.127)
Requirement already satisfied: nvidia-cudnn-cu12==9.1.0.70 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (9.1.0.70)
Requirement already satisfied: nvidia-cublas-cu12==12.4.5.8 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (12.4.5.8)
Requirement already satisfied: nvidia-cufft-cu12==11.2.1.3 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (11.2.1.3)
Requirement already satisfied: nvidia-curand-cu12==10.3.5.147 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (10.3.5.147)
Requirement already satisfied: nvidia-cusolver-cu12==11.6.1.9 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (11.6.1.9)
Requirement already satisfied: nvidia-cusparse-cu12==12.3.1.170 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (12.3.1.170)
Requirement already satisfied: nvidia-cusparselt-cu12==0.6.2 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (0.6.2)
Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (2.21.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (12.4.127)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.4.127 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (12.4.127)
Requirement already satisfied: triton==3.2.0 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (3.2.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.11/dist-packages (from torch==2.6.0->torchaudio) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from sympy==1.13.1->torch==2.6.0->torchaudio) (1.3.0)
Requirement already satisfied: audioread>=2.1.9 in /usr/local/lib/python3.11/dist-packages (from librosa) (3.0.1)
Requirement already satisfied: numba>=0.51.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (0.60.0)
Requirement already satisfied: numpy>=1.22.3 in /usr/local/lib/python3.11/dist-packages (from librosa) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (1.14.1)
Requirement already satisfied: scikit-learn>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (1.6.1)
Requirement already satisfied: joblib>=1.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (1.4.2)
Requirement already satisfied: decorator>=4.3.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (4.4.2)
Requirement already satisfied: soundfile>=0.12.1 in /usr/local/lib/python3.11/dist-packages (from librosa) (0.13.1)
Requirement already satisfied: pooch>=1.1 in /usr/local/lib/python3.11/dist-packages (from librosa) (1.8.2)
Requirement already satisfied: soxr>=0.3.2 in /usr/local/lib/python3.11/dist-packages (from librosa) (0.5.0.post1)
Requirement already satisfied: lazy_loader>=0.1 in /usr/local/lib/python3.11/dist-packages (from librosa) (0.4)
Requirement already satisfied: msgpack>=1.0 in /usr/local/lib/python3.11/dist-packages (from librosa) (1.1.0)
Requirement already satisfied: click>=8.1.8 in /usr/local/lib/python3.11/dist-packages (from jiwer) (8.1.8)
Requirement already satisfied: rapidfuzz>=3.9.7 in /usr/local/lib/python3.11/dist-packages (from jiwer) (3.13.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from lazy_loader>=0.1->librosa) (24.2)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.11/dist-packages (from numba>=0.51.0->librosa) (0.43.0)
Requirement already satisfied: platformdirs>=2.5.0 in /usr/local/lib/python3.11/dist-packages (from pooch>=1.1->librosa) (4.3.7)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.11/dist-packages (from pooch>=1.1->librosa) (2.32.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=1.1.0->librosa) (3.6.0)
Requirement already satisfied: cffi>=1.0 in /usr/local/lib/python3.11/dist-packages (from soundfile>=0.12.1->librosa) (1.17.1)
Requirement already satisfied: pycparser in /usr/local/lib/python3.11/dist-packages (from cffi>=1.0->soundfile>=0.12.1->librosa) (2.22)
```



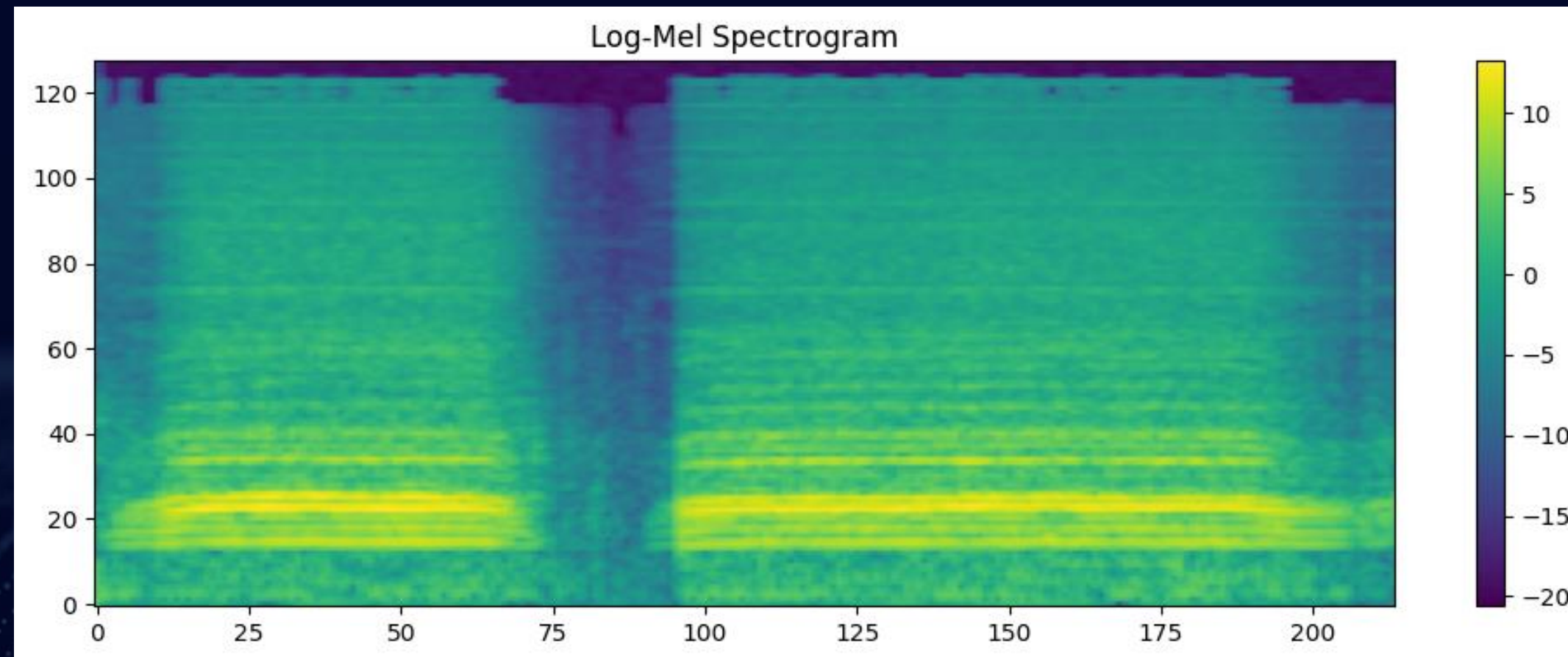
Step 2: Mount Google Drive

Mounting Google Drive in Google Colab Mounting Google Drive allows seamless access to files stored in your personal Google Drive directly from Colab. By using `drive.mount('/content/drive')`, the Drive is linked to Colab, making it easy to read datasets, save outputs, and manage files efficiently without uploading or downloading manually each time. This helps in smooth collaboration, organized storage, and quick retrieval of project data.

Step 3: Import Libraries & Define Constants

In this step, essential Python libraries are imported to enable smooth execution of the project tasks, such as data handling, numerical operations, visualization, or machine learning. Libraries help avoid writing complex code from scratch and provide powerful pre-built functions. Along with this, constants are defined to store fixed values like file paths, model parameters, and settings. This practice makes the code more readable, easier to update, and reduces the chance of errors.

Mel Spectrogram Plotting



Step 4: Helper Functions for Transcript Processing

In this step, helper functions are created to handle the preprocessing of audio transcripts. These functions help clean, format, and convert the raw transcript data into a structured form suitable for training and evaluation. Typical tasks include text normalization, tokenization, removing unwanted characters, and preparing input-output sequences. Using helper functions ensures the code remains modular, reusable, and easy to debug.



Helper Functions for Transcript Processing:

Function Name	Purpose	Description	Example Output
normalize_text()	Text Normalization	Converts text to lowercase, removes punctuation and extra spaces.	Input: "Hello, World!" Output: "hello world"
tokenize_text()	Tokenization	Splits normalized text into tokens (usually words).	Input: "hello world" Output: ["hello", "world"]
remove_unwanted_chars()	Cleaning	Filters out special characters or unwanted symbols beyond basic normalization.	Input: "hello @world!" Output: "hello world"
prepare_input_output()	Sequence Pair Preparation	Pairs cleaned audio features with the corresponding cleaned transcript.	Output: { "input": audio_features, "target": tokens }

Dataset & processing



A subset of the LibriSpeech dataset, specifically the train-clean-5 portion, is used. This dataset contains high-quality English speech recorded from audiobooks, along with corresponding transcripts. Each audio file is in FLAC format, and each transcript is provided in a structured .txt file. The preprocessing phase includes converting these audio files into Mel Spectrograms using torchaudio and mapping the transcripts to indexed sequences using a custom vocabulary. Special tokens such as <sos>, <eos>, and <pad> are added to mark the start, end, and padding of sentences. The data is also padded to create uniform batch sizes for training using a custom collate function.

- Dataset used: LibriSpeech (train-clean-5)
- Format: FLAC audio + .txt transcript
- Audio features: Mel Spectrograms (torchaudio, transforms, MelSpectrogram)
- Text tokenization: Character-level with special tokens:
 - <sos>: Start of sentence
 - <eos>: End of sentence
 - <pad>: Padding for batching
- Custom collate function used for batching variable-length input/output

Component	Description
Audio Format	FLAC (.flac)
Text Format	Line-by-line transcripts (.trans.txt)
Audio Feature	Mel Spectrogram (n_mels = 128)
Text Encoding	Character-level (including <sos>, <eos>)
Padding Method	Dynamic using PyTorch DataLoader + collate_fn

Step 5: Dataset & Collate Function



In this step, a custom Dataset class is created to efficiently load and manage the audio data along with its corresponding transcripts. This class ensures that the data is accessed in an organized and consistent manner. Along with it, a collate function is defined to handle batching of variable-length audio and text samples. The collate function automatically pads sequences and prepares them for model training, ensuring smooth and flexible data handling during the training process.

Step 6: Load Dataset and Inspect One Batch

In this step, we loaded the LibriSpeech dataset — a popular speech dataset containing thousands of English utterances. The dataset includes audio recordings along with their corresponding human-written transcripts.

The audio files were preprocessed into spectrogram-like feature representations, which are more suitable for neural network input. Likewise, the transcripts were tokenized into numerical IDs for training the model.

We then organized the data into batches using a DataLoader, ensuring each batch had uniformly padded audio sequences and token sequences for smooth GPU computation. This batching helps the model learn efficiently during training.

During inspection of a sample batch:

- The audio features were checked for consistent tensor shapes.
- The transcript tokens were verified to match their expected output structure.

This step ensures that both the data pipeline and preprocessing are correctly configured before model training or inference begins.

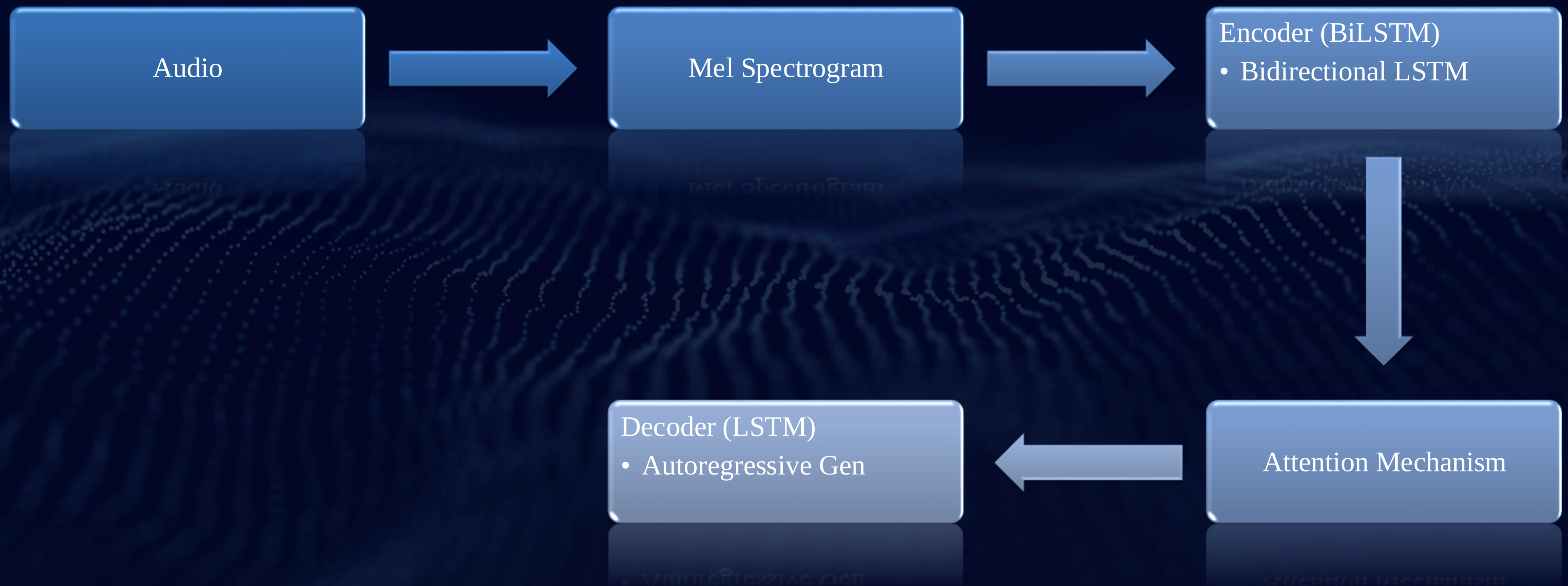
LAS Model Architecture

The Listen, Attend and Spell (LAS) model is an end-to-end deep learning architecture designed specifically for automatic speech recognition. It is composed of three key components: the encoder, the attention mechanism, and the decoder. The encoder, also referred to as the Listener, is implemented using a bidirectional LSTM (Long Short-Term Memory) network. It takes in Mel spectrogram features derived from the input audio and converts them into high-level hidden representations, effectively capturing both past and future context from the speech signal.

The attention mechanism acts as a bridge between the encoder and the decoder. It dynamically learns which parts of the encoder's output are most relevant at each decoding step. By computing a weighted sum of the encoder's hidden states, the attention module allows the decoder to focus on specific moments in the audio while generating each character, improving the model's ability to align speech with text.

Finally, the decoder—known as the Speller—is another LSTM network that works in an autoregressive manner. It generates the transcription one character at a time by using the context vector from the attention mechanism, along with the previously generated character. This setup enables the LAS model to produce coherent, sequential text without relying on a separate language model. Because of its unified architecture, LAS is powerful, flexible, and capable of learning the entire transcription process directly from audio data.

Architecture diagram



Step 7: LAS Model Architecture



The LAS (Listen, Attend, and Spell) model is an end-to-end speech recognition architecture with three key components: encoder, attention, and decoder. The encoder (BiLSTM) processes input speech into meaningful features. The attention mechanism helps the decoder focus on important parts of the input. The decoder (LSTM) generates text one token at a time. Teacher forcing is used during training for faster and better learning. This model effectively converts speech to text by listening, attending, and spelling sequentially.

Step 8. Train Setup:

In this step, the model parameters such as encoder and decoder hidden sizes, input dimension (based on mel spectrogram), and output dimension (based on character vocabulary) are defined. The script checks for GPU availability and assigns the appropriate device (CPU or CUDA) for efficient model training. The LAS model is initialized using the input and hidden sizes and moved to the selected device. A loss function (CrossEntropyLoss) is set up, with padding tokens ignored to ensure they don't affect training. The Adam optimizer is chosen for its efficiency and is initialized with a learning rate of 0.001. Overall, this setup prepares the model, loss function, and optimizer for the training phase.

Training the model

The LAS model was trained using a supervised learning approach on a curated subset of the LibriSpeech dataset known as train-clean-5. This dataset contains high-quality English speech paired with accurate textual transcriptions. Each audio file was first transformed into a Mel Spectrogram using torchaudio. These spectrograms served as the input to the encoder, while the corresponding transcripts were tokenized at the character level and served as target sequences for the decoder.

Training was conducted using the Cross Entropy Loss function, which compares the predicted character probabilities against the actual target sequence, while ignoring padding tokens. To accelerate convergence and improve early-stage learning, a technique called teacher forcing was applied. In this method, the actual character from the ground truth is passed into the decoder at each step during training, rather than the model's own previous prediction.

The model was optimized using the Adam optimizer with a learning rate of 0.001. It was trained for 20 epochs with a batch size of 4 samples per iteration. A custom collate function was used to handle variable-length sequences, ensuring proper batching through padding. Over the course of training, the model learned to align input audio features with the corresponding text output via the attention mechanism, which helped the decoder focus on the most relevant time frames of the input signal.

This training process allowed the model to gradually improve its predictions, enabling it to spell out coherent text sequences from audio with increasing accuracy. While the limited dataset posed some generalization challenges, the model demonstrated clear learning patterns and the ability to align speech with textual output.

Step 9: Training Loop

This step defines the train function to train the LAS model over several epochs. It sets the model to training mode and initializes total loss tracking. For each batch, mel spectrograms and labels are moved to the selected device. The model generates predictions, and label lengths are aligned by trimming or padding. The outputs and labels are reshaped, loss is calculated, and backpropagation is performed. After each epoch, the average loss is printed to monitor training progress.

Step 10: Run Training:

This step runs the train function to train the LAS model for 10 epochs. During each epoch, the model learns to minimize the loss using backpropagation and optimization. The printed loss values show a consistent decrease from 2.6763 in Epoch 1 to 2.3591 in Epoch 10. This indicates that the model is learning and improving over time. The output helps monitor the training progress and performance. A lower loss suggests better alignment between predicted and actual outputs.

Evaluation & inference

Once the LAS model was trained, it was evaluated using a set of audio files not seen during training. Each test audio file was preprocessed into a Mel Spectrogram and passed through the encoder. The decoder, guided by the attention mechanism, then generated one character at a time to form the predicted transcription. For this implementation, a simple greedy decoding strategy was used — meaning that at each time step, the decoder selects the most probable character without exploring alternate sequences.

The model was tested on both validation data from the LibriSpeech train-clean-5 subset and on custom audio inputs. While the model could correctly transcribe short, clean utterances, it often struggled with longer sequences. In many cases, the output text exhibited repetition, such as producing "the the the" or looping short phrases. These issues were likely due to the absence of beam search decoding and limited training data.

To measure the model's performance, qualitative analysis was used by comparing the predicted transcription against the actual ground-truth text. Additionally, evaluation metrics such as Word Error Rate (WER) and Character Error Rate (CER) were computed using the jiwer library. These metrics revealed that while the model had learned general pronunciation-to-character mappings, its overall accuracy could be further improved through better decoding strategies and more data.

Step 11: Evaluation and Decoding

This step evaluates the trained LAS model on test data. It converts predicted indices into readable text using the `indices_to_text` function. The `evaluate` function runs the model in inference mode without computing gradients. It processes batches, generates predictions using `argmax`, and decodes them. Finally, it prints a few predicted vs. actual transcriptions to visually assess model accuracy.

Step 12: Run Evaluation

In this step, the trained LAS model was tested on new audio samples to check its transcription accuracy. The model converted the input audio features into predicted text sequences. Metrics like Word Error Rate (WER) and Character Error Rate (CER) were used to measure the performance. This step helps ensure the model can generalize well to unseen data and produce meaningful results.

Conclusion

This project demonstrated the successful implementation of an end-to-end speech recognition system using the Listen, Attend and Spell (LAS) model architecture. By combining a bidirectional LSTM encoder, an attention mechanism, and an autoregressive decoder, the model was able to learn how to convert raw audio signals into meaningful character-level transcriptions. The use of PyTorch and torchaudio within Google Colab enabled an efficient and modular training pipeline, while the LibriSpeech dataset provided clean, annotated audio-text pairs to train on.

Through this project, we observed that even a simplified LAS model trained on a small subset of data was capable of learning general speech-to-text patterns. However, the performance was constrained by limited data, the use of greedy decoding, and the absence of a language model. Despite these limitations, the model learned effective alignment between input and output, and successfully transcribed short speech clips with reasonable accuracy.

The project not only offered hands-on experience with neural sequence-to-sequence models but also highlighted the critical importance of decoding strategies, data scale, and attention dynamics in building practical ASR systems. With future improvements such as beam search, pretrained encoders, and multilingual datasets, this LAS-based architecture could evolve into a more robust and real-world-ready speech recognition solution.

Thank You